

**PES UNIVERSITY, BANGALORE**

Department of Computer Science and Engineering

**JACKFRUIT MINIPROJECT**

Subject Code: UE24CS341A — Software Engineering, V Semester

# **Software Requirements Specification**

for

## **Simple Shell — POSIX-Compliant Command Line Interpreter in C**

Version 1.0 — Approved

Group: TEAM 10

Anshul Poovaiah K · Archana Shivakumar · Ashlesh R V · Atishay Jain

Date Created: September 7, 2026

# Table of Contents

---

|                                                               |           |
|---------------------------------------------------------------|-----------|
| Revision History .....                                        | ii        |
| <b>1. Introduction .....</b>                                  | <b>3</b>  |
| 1.1 Purpose.....                                              | 3         |
| 1.2 Intended Audience and Reading Suggestions .....           | 3         |
| 1.3 Product Scope .....                                       | 3         |
| 1.4 References .....                                          | 3         |
| <b>2. Overall Description .....</b>                           | <b>4</b>  |
| 2.1 Product Perspective .....                                 | 4         |
| 2.2 Product Functions .....                                   | 4         |
| 2.3 User Classes and Characteristics .....                    | 4         |
| 2.4 Operating Environment .....                               | 4         |
| 2.5 Design and Implementation Constraints .....               | 4         |
| 2.6 Assumptions and Dependencies .....                        | 5         |
| <b>3. External Interface Requirements .....</b>               | <b>5</b>  |
| 3.1 User Interfaces .....                                     | 5         |
| 3.2 Software Interfaces .....                                 | 5         |
| 3.3 Communications Interfaces .....                           | 5         |
| 3.4 Hardware Interfaces .....                                 | 5         |
| <b>4. Analysis Models .....</b>                               | <b>6</b>  |
| 4.1 System Context Diagram .....                              | 6         |
| 4.2 Use Case Model .....                                      | 6         |
| 4.3 Command Execution Activity Flow .....                     | 7         |
| 4.4 Process / Job Lifecycle State Model .....                 | 7         |
| 4.5 Lexer Token Recognition Model.....                        | 7         |
| <b>5. System Features.....</b>                                | <b>8</b>  |
| 5.1 – 5.8 (one per functional feature, FR-001 – FR-010) ..... | 8         |
| <b>6. Other Nonfunctional Requirements .....</b>              | <b>10</b> |
| 6.1 Performance Requirements.....                             | 10        |
| 6.2 Safety Requirements .....                                 | 10        |
| 6.3 Security Requirements .....                               | 10        |
| 6.4 Software Quality Attributes .....                         | 11        |
| 6.5 Business Rules .....                                      | 11        |
| <b>7. Other Requirements .....</b>                            | <b>12</b> |
| <b>Appendix A: Glossary.....</b>                              | <b>12</b> |
| <b>Appendix B: Field Layouts .....</b>                        | <b>13</b> |
| <b>Appendix C: Requirement Traceability Matrix .....</b>      | <b>13</b> |

# Revision History

---

| Name                         | Date        | Reason For Changes                                        | Version |
|------------------------------|-------------|-----------------------------------------------------------|---------|
| Archana Shivakumar (Team 10) | 07-Sep-2026 | Initial SRS Phase-1 draft prepared from approved Synopsis | 1.0     |

## 1. Introduction

---

### 1.1 Purpose

This document specifies the software requirements for Simple Shell — a POSIX-compliant command-line interpreter developed in C — as undertaken by Team 10 (Project ID: SH-2026-T10) for the UE24CS341A Software Engineering Jackfruit Miniproject. This SRS covers the complete scope of the Simple Shell executable: the interactive REPL, lexical parsing and environment expansion, process execution, multi-stage pipelines, I/O redirection, job control and signal handling, built-in commands, and persistent history/alias management. This is Version 1.0 of the SRS, corresponding to Jackfruit Phase-1 (Requirements).

### 1.2 Intended Audience and Reading Suggestions

This document is intended for four audiences:

- Team 10 developers (Anshul, Archana, Ashlesh, Atishay) — to derive module design and implementation tasks directly from Section 5 (System Features).
- Course faculty and evaluators (UE24CS341A) — to verify requirement measurability, traceability, and coverage against the Jackfruit grading rubric.
- Testers — to derive unit, integration, and system test cases directly from the numbered FR/NFR/SEC requirements and the Validation Specification (Section 6 and the companion Validation Spec document).
- Future maintainers — to understand system scope and constraints without reading source code first.

Readers primarily interested in what the shell does should read Sections 2 and 5. Readers interested in engineering constraints should read Sections 2.5, 2.6, 6, and 7. Section 4 (Analysis Models) and Appendix C (Traceability Matrix) connect requirements to the design and implementation artifacts that follow in later Jackfruit phases.

### 1.3 Product Scope

Simple Shell is a lightweight, modular command-line interpreter for Unix/Linux systems, built to demonstrate industrial-grade systems programming practice rather than to replace a production shell such as bash or zsh. It reads user input through a Read-Eval-Print Loop, tokenizes and expands it, resolves and executes binaries via POSIX system calls, and provides pipelines, redirection, background job control, built-ins, and persistent history. It benefits students by providing a hands-on demonstration of process management, IPC, and signal handling; it benefits Interactive Developers and Script Automation Runners (Section 2.3) by giving them a small, auditable, memory-safe interpreter. The project directly supports Team 10's course objective of demonstrating full-lifecycle Agile software engineering (Requirements → Design → Implementation → Validation → Demo) on a systems-level C codebase.

### 1.4 References

- PES University, Department of CSE — "Jackfruit Miniproject: Software Engineering Project Guidelines", Subject Code UE24CS341A.
- Team 10 — "Simple Shell (CLI): Deliverable 1 — Synopsis / Project Proposal", v1.0, approved.
- IEEE Std 830 — style basis for the SRS template supplied by the course (Jackfruit Phase-1 SRS Template).

- The Open Group Base Specifications Issue 7 (POSIX.1-2008) — system call and shell behaviour reference (fork, execvp, pipe, dup2, sigaction, waitpid).
- GNU Bash Reference Manual — used as the black-box behavioural comparison baseline referenced in NFR verification.

## 2. Overall Description

---

### 2.1 Product Perspective

Simple Shell is a new, self-contained, standalone product — it is not a follow-on to an existing Team 10 system. It sits directly above the Linux kernel as a user-space process: it does not modify, replace, or require changes to the kernel, and it coexists with the system's default login shell rather than displacing it. Its only external system dependencies are the POSIX system-call interface and the standard C library (glibc). The diagram below (Section 4.1) shows Simple Shell's position between the human/script user and the kernel-managed process and file-system resources it orchestrates.

### 2.2 Product Functions

At a high level, Simple Shell lets a user or script:

- Interact with the system through a live, formatted command prompt (REPL).
- Enter commands using shell-standard syntax: quoting, escaping, environment variables, and the previous exit status (\$?).
- Execute any binary on PATH as a child process and retrieve its exit status.
- Chain arbitrarily many commands into a pipeline using |.
- Redirect standard input, standard output, and standard error to and from files.
- Run commands in the background with &, and manage them via jobs, fg, and bg.
- Use built-in commands (cd, pwd, echo, export, unset, exit) that must run in the parent shell's own process context.
- Recall history across sessions and define command aliases.

Section 5 (System Features) expands each of these into numbered, testable functional requirements.

### 2.3 User Classes and Characteristics

| User Class                   | Frequency / Privilege                          | Characteristics                                                                                                                                                                                                       |
|------------------------------|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interactive Developer / User | Frequent; standard shell (non-root) privileges | Uses the shell interactively for most of a session; relies heavily on pipelines, redirection, job suspension (Ctrl+Z) and history recall. Highest-priority user class — most requirements are written for this class. |
| Script / Automation Runner   | Frequent, non-interactive; standard privileges | Feeds pre-written command sequences via stdin or a script file. Needs deterministic exit codes and fast, predictable startup rather than interactive conveniences such as history or aliases.                         |
| Systems Evaluator / Faculty  | Occasional, administrative/audit access        | Exercises boundary conditions (deep pipelines, malformed input, permission-denied files) and reviews Valgrind/ASan/static-analysis evidence rather than everyday shell usage.                                         |

## 2.4 Operating Environment

- Operating System: Ubuntu 22.04 LTS or Debian 12, Linux kernel 5.15 or later (POSIX.1-2008 compliant).
- Language / Standard: C11, compiled with GCC 11+ using `-Wall -Wextra -Werror -pedantic`.
- Build system: GNU Make.
- Terminal: any POSIX-compliant pseudo-terminal (pty) / TTY device; no GUI or windowing system dependency.
- Containerized target: Docker (for reproducible CI/CD builds and grading), in addition to bare-metal/VM execution.

## 2.5 Design and Implementation Constraints

- Course constraint: development must follow Agile/Scrum with GitHub (and Jira or equivalent) actively used for backlog and sprint tracking; every team member must contribute implementation commits (checked via Git history).
- Language constraint: implementation is restricted to C (POSIX system calls) — no use of higher-level runtimes (Python/Lua) or third-party shell libraries for core parsing or execution logic.
- Memory constraint: dynamic allocation must be explicitly freed; zero leaks and zero invalid accesses are mandatory acceptance criteria (Valgrind, AddressSanitizer), not optional quality goals.
- Concurrency constraint: all pipeline file descriptors must be explicitly closed in every process that does not need them, to avoid deadlocks and descriptor leaks.
- Signal-handler constraint: signal handlers may only call async-signal-safe functions (e.g., `waitpid`, `write`) — no `printf` or `malloc` inside a handler.
- Toolchain constraint: CI/CD must run on GitHub Actions or Jenkins and must include compilation, static analysis (Cppcheck/Clang-Tidy), unit tests, and a Valgrind pass on every pull request.

## 2.6 Assumptions and Dependencies

- Assumes the target machine already provides a POSIX-compliant kernel and a standard glibc toolchain; Simple Shell does not install or manage these.
- Assumes evaluators will run the shell in a genuine terminal/pty (interactive job-control signals such as SIGTSTP behave differently or are unavailable when stdin is not a TTY).
- Depends on third-party open-source tooling only for development/QA — Unity or Criterion (unit testing), Valgrind, Cppcheck/Clang-Tidy, Docker — none of which are linked into the shipped shell binary itself.
- Assumes each of the four "in scope" binaries a user invokes (`ls`, `grep`, `cat`, etc.) are themselves correctly installed on PATH; Simple Shell is not responsible for the correctness of external programs it launches.
- If GitHub Actions minute quotas or Jira/GitHub Projects access change mid-project, the team's Agile tracking evidence (Section 2.5) would need to move to an equivalent tool — this is treated as a project risk, not a requirements change.

# 3. External Interface Requirements

---

## 3.1 User Interfaces

Simple Shell exposes a single, text-only command-line interface — there is no graphical interface (explicitly out of scope; see Synopsis Section "Out of Scope"). Interface characteristics:

- Prompt format: `[user]@[hostname]:[cwd]$`, re-rendered after every command completes.
- Line editing: raw line input via `getline()`; arrow-key history recall is a Medium-priority convenience (FR-010), not a baseline requirement.

- Error message convention: all shell-generated errors are prefixed `simple_shell:` to distinguish them from the output of executed child programs, and are written to `stderr`, never `stdout`.
- Exit status convention: the shell exposes the previous command's exit status through `$?`, following standard POSIX shell convention (0 = success, 1–125 = command-defined failure, 127 = command not found, 128+n = terminated by signal n).

## 3.2 Software Interfaces

| Interface                                           | Direction                                                 | Purpose                                                                                                                                                                                                                                                                                                      |
|-----------------------------------------------------|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Linux kernel (POSIX system calls)                   | Bidirectional                                             | <code>fork</code> , <code>execvp/execve</code> , <code>pipe</code> , <code>dup2</code> , <code>waitpid</code> , <code>sigaction</code> , <code>open/close</code> , <code>chdir</code> — the shell's entire execution and I/O model is built directly on these calls; no intermediate library abstracts them. |
| Filesystem ( <code>~/simple_shell_history</code> )  | Read on startup, write on exit                            | Serializes and restores persistent command history. File permissions restricted to mode 0600 (SEC-006).                                                                                                                                                                                                      |
| Executed child binaries (any PATH-resolved program) | Bidirectional ( <code>stdin/stdout/stderr</code> via fds) | Simple Shell does not interpret child output; it only wires up file descriptors (pipes/redirections) before <code>execvp</code> and collects the exit status afterward.                                                                                                                                      |
| Unit test framework (Unity / Criterion)             | Build-time only                                           | Links against internal modules (lexer, expander, parser) for white-box unit testing; not present in the shipped release binary.                                                                                                                                                                              |

## 3.3 Communications Interfaces

Simple Shell has no network communication requirement in this phase — remote/SSH daemon capability and socket streaming are explicitly out of scope (Synopsis, Out of Scope list). The only "communication" channels are local: standard input/output/error streams and POSIX signals (`SIGINT`, `SIGTSTP`, `SIGCHLD`) delivered by the kernel to the shell process.

## 3.4 Hardware Interfaces

Simple Shell interacts with hardware only indirectly, through the kernel's TTY/pty abstraction. It requires a controlling terminal to receive keyboard-generated signals (`Ctrl+C` → `SIGINT`, `Ctrl+Z` → `SIGTSTP`) correctly; when run with input piped from a non-TTY source (a script or CI harness), job-control signal delivery is not applicable and the shell falls back to straight sequential execution.

# 4. Analysis Models

---

## 4.1 System Context Diagram

Simple Shell sits between two human/script-driven actors and the Linux kernel, which in turn manages the child processes the shell spawns:

| Actor                        | Goal / Need                                                        | Major Actions                                                                                                       |
|------------------------------|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Interactive Developer / User | Execute OS utilities, chain tools, manage running jobs             | Input commands via prompt; configure aliases/environment; pipe utilities; redirect streams; manage background tasks |
| Batch / Automation Runner    | Execute sequential scripted commands with deterministic exit codes | Feed scripts via <code>stdin</code> or file; process commands sequentially; terminate with status codes             |

| Actor                          | Goal / Need                                                     | Major Actions                                                                                                      |
|--------------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Linux Kernel / Child Processes | Spawn isolated process spaces, execute binaries, manage streams | Create PIDs (fork); replace memory image (execve); manage pipe buffers; deliver signals (SIGINT, SIGTSTP, SIGCHLD) |

## 4.2 Use Case Model

Seven use cases cover the full functional surface described in Section 5. All are initiated by the Interactive Developer / User actor; the Linux Kernel is a supporting actor in each.

| ID    | Use Case                         | Precondition                                 | Expected Outcome                                                                                        |
|-------|----------------------------------|----------------------------------------------|---------------------------------------------------------------------------------------------------------|
| UC-01 | Launch Shell Session             | Executable available on PATH                 | REPL prompt displayed; history loaded from disk.                                                        |
| UC-02 | Execute External Command         | Shell running; binary exists on PATH         | Process spawned, executed, output displayed, exit status collected.                                     |
| UC-03 | Execute Multi-Stage Pipeline     | Shell running; valid piped commands          | Commands run concurrently; streams connected via pipes; final status captured.                          |
| UC-04 | Redirect Stream to File          | Shell running; valid target path             | Output/error written to file, or input read from source file.                                           |
| UC-05 | Launch and Manage Background Job | Shell running; command suffixed with &       | Job assigned a Job ID; prompt returns immediately; job listed in jobs.                                  |
| UC-06 | Execute Built-in Command         | Shell running; built-in keyword entered      | Internal shell state modified (e.g., directory changed, variable set) without spawning a child process. |
| UC-07 | Recall History and Use Aliases   | Shell running; history/alias table populated | Previous commands listed/recalled; alias expanded to its full command line before execution.            |

## 4.3 Command Execution Activity Flow

The dominant end-to-end activity flow, shared by UC-02 through UC-06, is:

1. Read line (getline) → 2. Tokenize (lexer) → 3. Expand variables/\$? (expander) → 4. Parse into Pipeline/Command structs (parser) → 5. Is first token a built-in? → 5a. Yes: execute in parent process, skip to step 8 → 5b. No: continue → 6. Resolve binary via PATH, fork, apply redirection/pipe dup2, execvp → 7. Parent waitpid(s) (foreground) or registers a Job and returns immediately (background) → 8. Update \$? → 9. Re-render prompt.

## 4.4 Process / Job Lifecycle State Model

Every foreground or background child process (and, correspondingly, every tracked Job) moves through a small state machine:

| State        | Entered When                                            | Exits To                                                       |
|--------------|---------------------------------------------------------|----------------------------------------------------------------|
| RUNNING (FG) | fork() succeeds and the command was not suffixed with & | TERMINATED on exit, or STOPPED on SIGTSTP                      |
| RUNNING (BG) | fork() succeeds and the command was suffixed with &     | TERMINATED on exit (reaped via SIGCHLD), or STOPPED on SIGTSTP |

| State                        | Entered When                                               | Exits To                                                                    |
|------------------------------|------------------------------------------------------------|-----------------------------------------------------------------------------|
| STOPPED                      | SIGTSTP delivered to the foreground process group (Ctrl+Z) | RUNNING (BG) via bg, RUNNING (FG) via fg, or TERMINATED if killed           |
| TERMINATED (Zombie → Reaped) | Child calls exit() or is killed by a signal                | Removed from the Job table once waitpid()/SIGCHLD-handler reaping completes |

## 4.5 Lexer Token Recognition Model

The tokenizer (FR-002) is a single-pass character scanner with three context flags — `in_single_quote`, `in_double_quote`, `in_escape` — that determine whether whitespace and operator characters (`|`, `<`, `>`, `>>`, `2>`, `&`) are treated as token separators or as literal text. A token boundary is only emitted when all three flags are false; this is elaborated into a full DFA diagram in the Software Design Document (Jackfruit Phase-2).

## 5. System Features

---

Functional requirements are organized below by system feature, matching the eight functional features enumerated in the approved Synopsis (Section 1.5). Requirement IDs (FR-00x) are kept identical to those used in the Synopsis and will carry forward unchanged into the Architecture, Design, and Requirement Traceability Matrix (Appendix C) in later Jackfruit phases, so that every requirement remains traceable end-to-end.

### 5.1 Interactive REPL & Dynamic Prompt

**Description and Priority:** Priority — High.

#### Stimulus/Response Sequences

- User launches the compiled shell binary from a terminal.
- Shell reads the current username, hostname, and working directory and renders the prompt.
- User types a command line and presses Enter; the shell reads it via `getline()` and re-renders the prompt once execution completes.

#### Functional Requirements

**FR-001:** The system shall display an interactive prompt indicating the current user, hostname, and working directory, and read arbitrary-length user input lines.

### 5.2 Lexical Analysis & Environment Substitution

**Description and Priority:** Priority — High.

#### Stimulus/Response Sequences

- User enters a line containing quoted strings, escape characters, and/or `$VAR` / `$?` tokens.
- The lexer tokenizes the line, honoring quote and escape context, before the parser or expander sees it.
- The expander substitutes each `$VAR` with its environment value (or empty string if undefined) and `$?` with the previous command's exit status.

#### Functional Requirements

**FR-002:** The system shall tokenize the input line into discrete command arguments, correctly parsing single quotes, double quotes, and escaped characters.

**FR-003:** The system shall expand environment-variable tokens prefixed with `$` (e.g., `$HOME`, `$PATH`) and shall expand `$?` to the exit status of the previous foreground command.

### 5.3 External Command Execution

**Description and Priority:** Priority — High.

#### Stimulus/Response Sequences

- User enters a command that is not a recognized built-in.
- The shell scans directories listed in PATH to locate a matching executable.
- The shell forks a child process, replaces its image via `execvp`, and the parent waits on the child via `waitpid` to collect its exit status.

#### Functional Requirements

**FR-004:** The system shall locate external binaries by scanning directories listed in PATH, fork a child process, execute the resolved binary via `execvp`, and collect its termination status via `waitpid`.

### 5.4 Multi-Stage Pipeline IPC

**Description and Priority:** Priority — High.

#### Stimulus/Response Sequences

- User enters two or more commands separated by `|`.
- The shell creates N-1 pipes for N pipeline stages, forks one child per stage, and wires each stage's stdout to the next stage's stdin via `dup2`.
- Every process closes all pipe file descriptors it does not need, then the parent waits for all stages and reports the final stage's exit status.

#### Functional Requirements

**FR-005:** The system shall support multi-stage command pipelines (`cmd1 | cmd2 | ... | cmdN`) of arbitrary depth, connecting each stage's output to the next stage's input using POSIX pipes, and shall close all unused file descriptors in every process to prevent deadlocks.

### 5.5 File Stream Redirection

**Description and Priority:** Priority — High.

#### Stimulus/Response Sequences

- User appends `<`, `>`, `>>`, or `2>` plus a filename to a command.
- The shell opens the target file with the mode implied by the operator (read-only for `<`, write-truncate for `>`, write-append for `>>`, write-truncate on fd 2 for `2>`).
- The shell duplicates the opened file descriptor onto the appropriate standard stream via `dup2` before executing the command, and restores the original descriptors afterward for built-ins that don't fork.

#### Functional Requirements

**FR-006:** The system shall redirect standard input (`<`), standard output truncate (`>`), standard output append (`>>`), and standard error (`2>`) to specified files, aborting the command with a non-zero exit code if the target file cannot be opened.

### 5.6 Job Control & Asynchronous Background Processing

**Description and Priority:** Priority — High.

#### Stimulus/Response Sequences

- User appends `&` to a command line.

- The shell forks the command, immediately returns control to the prompt without waiting, and assigns and prints an incrementing Job ID and PID.
- On SIGCHLD delivery, a non-blocking reap loop (waitpid(-1, &status, WNOHANG)) clears completed background jobs; jobs, fg, and bg let the user inspect and move jobs between foreground and background.
- On Ctrl+C or Ctrl+Z, the shell forwards SIGINT/SIGTSTP to the active foreground child while the parent shell itself remains alive and responsive.

#### Functional Requirements

**FR-007:** The system shall execute commands ending in & asynchronously in the background, assign an incrementing Job ID, and maintain a responsive parent prompt without blocking.

**FR-008:** The system shall catch SIGINT (Ctrl+C) and SIGTSTP (Ctrl+Z) and forward them to the active foreground child process while preserving the parent shell process.

### 5.7 Shell Built-in Commands

**Description and Priority:** Priority — High.

#### Stimulus/Response Sequences

- User enters cd, pwd, echo, export, unset, or exit.
- The shell recognizes the leading token as a built-in before attempting PATH resolution.
- The built-in is executed directly inside the parent shell process (no fork), so that state-changing built-ins such as cd and export take effect in the shell itself rather than a short-lived child.

#### Functional Requirements

**FR-009:** The system shall implement parent-context built-in commands (cd, pwd, echo, export, unset, exit, jobs, fg, bg) without spawning a child process for any of them.

### 5.8 Session History & Alias Subsystem

**Description and Priority:** Priority — Medium.

#### Stimulus/Response Sequences

- Every accepted command line is appended to an in-memory history ring buffer during the session.
- On shell exit, the buffer is serialized to ~/.simple\_shell\_history (mode 0600); on the next launch, this file is read back into memory.
- User defines an alias via alias name='command'; on subsequent input, the shell substitutes the alias's expansion before tokenizing further.

#### Functional Requirements

**FR-010:** The system shall append executed commands to a history log, serialize entries to ~/.simple\_shell\_history on exit, load previous history on startup, and support user-defined command aliases via alias/unalias.

## 6. Other Nonfunctional Requirements

---

### 6.1 Performance Requirements

Rationale: because Simple Shell sits on the critical path of every command a user runs, dispatch overhead must be imperceptible even under heavy pipeline use.

| ID      | Category    | Measurable Requirement                                                                                                        | Verification Method      |
|---------|-------------|-------------------------------------------------------------------------------------------------------------------------------|--------------------------|
| NFR-001 | Performance | The shell shall parse and dispatch a simple external command with overhead under 2.0 milliseconds on standard POSIX hardware. | Benchmark timing harness |

## 6.2 Safety Requirements

"Safety" here means protecting the host system and the user's own session from damage caused by the shell's own defects (as distinct from Section 6.3, which covers protecting the shell from malicious input).

| ID      | Category             | Measurable Requirement                                                                                                                              | Verification Method           |
|---------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| NFR-003 | Concurrency / Safety | The shell shall cleanly reap all background child processes upon SIGCHLD delivery without creating lingering zombie processes.                      | Process status check (ps -ef) |
| NFR-004 | Reliability / Safety | The shell shall not crash or terminate when processing malformed input, missing binary paths, permission-denied files, or interrupted system calls. | Negative test suite           |

## 6.3 Security Requirements

| ID      | Security Requirement                                                                                                                                          | Validation Approach                            |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| SEC-001 | The shell shall dynamically allocate or bounds-check all user input buffers to prevent buffer overflow vulnerabilities.                                       | AddressSanitizer (-fsanitize=address) test run |
| SEC-002 | The shell shall sanitize and safely tokenize arguments to prevent command-injection vulnerabilities when passing arguments to execvp.                         | Static analysis & negative unit tests          |
| SEC-003 | The shell shall strictly respect Linux file-system permissions when opening redirection files, never elevating or bypassing file modes.                       | File permission test scripts                   |
| SEC-004 | The shell shall drop or avoid acquiring unneeded setuid/setgid privileges, adhering to the principle of least privilege.                                      | Security review & execution audit              |
| SEC-005 | The shell shall isolate parent environment modifications so that temporary subshell changes do not corrupt the parent environment unless explicitly exported. | Unit tests for export/unset                    |
| SEC-006 | The shell shall restrict history file permissions to mode 0600 (read/write by owner only) to safeguard sensitive command history.                             | File mode check (stat -c %a)                   |

## 6.4 Software Quality Attributes

| ID      | Category        | Measurable Requirement                                                                                                                                                           | Verification Method                           |
|---------|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| NFR-005 | Maintainability | The C codebase shall strictly separate lexical parsing, execution, redirection, and built-ins across distinct modules, with zero global-variable bleed across module boundaries. | Static analysis & code review                 |
| NFR-002 | Memory Safety   | The shell shall produce zero memory leaks and zero invalid memory accesses after 1,000 successive command executions.                                                            | valgrind --leak-check=full --error-exitcode=1 |
| NFR-006 | Portability     | The shell shall compile cleanly with gcc -Wall -Wextra -pedantic and execute identically on Ubuntu Linux and Debian Docker containers.                                           | Automated CI build matrix                     |

## 6.5 Business Rules

- Built-in commands that alter session state (cd, export, unset, exit, alias) must execute within the parent shell process, never a child.
- In a pipeline cmd1 | cmd2, every stage runs in a separate child process, and every unused pipe file descriptor must be closed in every process to prevent deadlocks.
- Delivery of SIGCHLD must trigger a non-blocking reap loop (waitpid(-1, &status, WNOHANG)) so completed child processes are promptly cleared.
- If a target redirection file cannot be opened (e.g., permission error), command execution must abort immediately and return a non-zero exit code.
- An empty command line, or one containing only whitespace, must simply reprint the prompt without error or exit-status change.

## 7. Other Requirements

---

- Portability: the shell shall be POSIX.1-2008 compliant and shall not rely on any Linux-only (non-POSIX) system call for core functionality.
- Packaging: the shell shall build and run correctly inside a Docker container (reproducible grading/demo environment), in addition to bare-metal/VM execution.
- Internationalization: out of scope for this phase — the shell assumes a single-byte, English-language locale (UTF-8 passthrough for arguments is not separately validated).
- Licensing / reuse: the codebase is an original academic implementation; any third-party code (test frameworks, sanitizers) is used strictly as a build/QA dependency and is not linked into the shipped binary.
- Database requirements: not applicable — Simple Shell has no persistent data store other than the flat-file history log described in FR-010.

## Appendix A: Glossary

---

| Term                 | Definition                                                                                                                                          |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| REPL                 | Read-Eval-Print Loop — the shell's core interactive cycle: read a line, evaluate/execute it, print the result, repeat.                              |
| PID                  | Process ID — the unique integer identifier the kernel assigns to a running process.                                                                 |
| PGID                 | Process Group ID — identifies a set of related processes (e.g., all stages of one pipeline) for signal delivery and job control.                    |
| Fork                 | The fork() system call, which creates a new child process that is a near-exact copy of the calling (parent) process.                                |
| Exec                 | The execvp()/execve() family of system calls, which replace a process's memory image with a new program.                                            |
| Pipe                 | A unidirectional, kernel-buffered communication channel (pipe()) connecting one process's output to another's input.                                |
| File Descriptor (fd) | A small non-negative integer handle a process uses to refer to an open file, pipe end, or other I/O resource.                                       |
| dup2                 | A system call that duplicates one file descriptor onto another descriptor number, used to rewire stdin/stdout/stderr for redirection and pipelines. |

| Term           | Definition                                                                                                                                                |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Signal         | An asynchronous notification delivered by the kernel to a process (e.g., SIGINT, SIGTSTP, SIGCHLD).                                                       |
| Zombie Process | A terminated child process whose exit status has not yet been collected by its parent via wait()/waitpid(); it remains in the process table until reaped. |
| TTY / pty      | The (pseudo-)terminal device through which a user's keyboard input and the shell's output are connected.                                                  |
| Job            | A background command (or pipeline) tracked by the shell with its own Job ID, distinct from its kernel PID.                                                |
| Built-in       | A command implemented and executed directly inside the shell's own process, rather than as a separately forked external binary.                           |
| WNOHANG        | A flag passed to waitpid() that makes it return immediately rather than blocking if no child has changed state — used for non-blocking zombie reaping.    |

## Appendix B: Field Layouts

Simple Shell has no database; the equivalent of "field layouts" for a systems project is the set of core in-memory C data structures each requirement operates on. These are frozen at Requirements sign-off and will be elaborated (with full struct definitions) in the Software Design Document.

| Structure | Field         | Data Type | Description                                            | Mandatory |
|-----------|---------------|-----------|--------------------------------------------------------|-----------|
| Command   | argv          | char**    | Null-terminated argument vector passed to execvp.      | Y         |
| Command   | argc          | int       | Number of string arguments.                            | Y         |
| Command   | input_file    | char*     | Source file path for < input redirection.              | N         |
| Command   | output_file   | char*     | Target file path for > or >> output redirection.       | N         |
| Command   | append_output | int       | Flag: 1 for append (>>), 0 for truncate (>).           | Y         |
| Pipeline  | command_count | int       | Number of stages chained by   in this pipeline.        | Y         |
| Pipeline  | is_background | int       | Flag: 1 if the command line terminates with &.         | Y         |
| Job       | job_id        | int       | Sequential integer index assigned to a background job. | Y         |
| Job       | pgid          | pid_t     | POSIX process group identifier for the job.            | Y         |
| Job       | status        | enum      | Operational status: RUNNING, STOPPED, or DONE.         | Y         |

## Appendix C: Requirement Traceability Matrix

Architecture, Design, Code, and full Test-Case references are marked TBD at this Requirements phase and will be populated during Jackfruit Phase-2 (Design) and Phase-3 (Implementation & Testing), per the course's phase-wise submission model. Requirement IDs below are frozen and will not be renumbered in later phases.

| Sl.No | Req. ID | Brief Description                      | Architecture Ref.  | Design Ref.        | Code File Ref. | Test Case ID | System ID |
|-------|---------|----------------------------------------|--------------------|--------------------|----------------|--------------|-----------|
| 1     | FR-001  | Interactive prompt & line ingestion    | TBD — Design Phase | TBD — Design Phase | TBD            | TBD          | TBD       |
| 2     | FR-002  | Lexical tokenization (quotes, escapes) | TBD — Design Phase | TBD — Design Phase | TBD            | TBD          | TBD       |
| 3     | FR-003  | Environment variable / \$? expansion   | TBD — Design Phase | TBD — Design Phase | TBD            | TBD          | TBD       |
| 4     | FR-004  | External binary resolution & execution | TBD — Design Phase | TBD — Design Phase | TBD            | TBD          | TBD       |
| 5     | FR-005  | Multi-stage pipeline IPC               | TBD — Design Phase | TBD — Design Phase | TBD            | TBD          | TBD       |
| 6     | FR-006  | File stream redirection                | TBD — Design Phase | TBD — Design Phase | TBD            | TBD          | TBD       |
| 7     | FR-007  | Background job execution               | TBD — Design Phase | TBD — Design Phase | TBD            | TBD          | TBD       |
| 8     | FR-008  | Signal handling (SIGINT/SIGTSTP)       | TBD — Design Phase | TBD — Design Phase | TBD            | TBD          | TBD       |
| 9     | FR-009  | Parent-context built-in commands       | TBD — Design Phase | TBD — Design Phase | TBD            | TBD          | TBD       |
| 10    | FR-010  | History serialization & aliases        | TBD — Design Phase | TBD — Design Phase | TBD            | TBD          | TBD       |
| 11    | NFR-001 | Command dispatch < 2ms                 | —                  | —                  | —              | TBD          | TBD       |
| 12    | NFR-002 | Zero memory leaks (Valgrind)           | —                  | —                  | —              | TBD          | TBD       |
| 13    | SEC-001 | Input buffer bounds checking           | TBD — Design Phase | TBD — Design Phase | TBD            | TBD          | TBD       |